Spring Boot 小游戏 Docker 容器化实验报告

记忆花园 memory-game6085

课程：2026 云计算技术与架构

姓名：周宇斌

班级：软件 2401

学号：20246085

实验日期：2026 年 10 月 21 日



图 1　记忆花园容器运行页面

本实验将 Vue3、Spring Boot 与 SQLite 记忆翻牌小游戏打包为 Docker 镜像，完成本机仓库发布、删除本地镜像后拉取和数据卷保留验证。正文记录原 Windows 实验，附录记录独立 Linux 云工作区的补充实验。v4 补齐已有资源创建截图和关键命令；两次实验的镜像版本、端口与结果分别保留。

目录

一 实验目的与任务要求.....3
二 实验环境与项目结构.....4
2.1 实验配置与项目目录.....4
2.2 环境启动与验证.....5
三 系统设计与持久化.....6
3.1 应用结构.....6
3.2 SQLite 数据设计.....6
3.3 容器数据卷.....6
四 本地构建与功能验证.....7
4.1 本地构建与自动验证.....7
4.2 容器中的真实游戏操作.....7
五 Docker 镜像构建与运行.....8
5.1 多阶段构建.....8
5.2 JAR 打包与镜像导出.....9
5.3 启动容器与接口检查.....10
六 镜像发布与拉取.....11
6.1 启动本机镜像仓库.....11
6.2 标记与推送镜像.....11
6.3 删除本地镜像后重新拉取.....12
七 容器重建与持久化验证.....14
7.1 记录重建前的数据.....14
7.2 使用拉取镜像创建新容器.....15
7.3 对比重建后的成绩.....16
八 实验结果与问题分析.....17
8.1 启动与端口问题.....17
8.2 实验总结.....17
九 参考资料与证据来源.....18
9.1 原 Windows 实验截图来源索引.....18
附录 A Linux 补充部署与复现.....19
A1 环境来源与独立资源配置.....19
A2 配套 JAR 打包为新镜像.....21
A3 容器运行与交付文件校验.....22
A4 实际游戏与成绩保存.....23
A5 本机仓库发布镜像.....24
A6 移除本地镜像并保留数据卷.....25
A7 从仓库重新拉取.....26
A8 重建容器并核对成绩保留.....27
A9 浏览器复核重建后的榜单.....28
A10 补充证据来源与版本对应.....29

一 实验目的与任务要求

本实验的目的是掌握 Spring Boot 应用容器化的基本过程，理解 Docker 镜像、容器、镜像仓库和持久化数据卷的作用，并通过实际命令和访问结果验证部署流程。

原始作业要求来自 raw/要求.png：在机器上部署 Docker 环境，将一个已有 Spring Boot 项目打包到 Docker 上发布，项目名称末尾加学号后四位；发布后演示拉取，所有过程截图放在一个 Word 文件中提交。

任务要求	本实验对应方式
项目名称包含学号后四位	memory-game6085，学号后四位为 6085
Spring Boot 项目打包发布	Windows 多阶段构建见第五章；Linux 配套 JAR 及镜像见附录 A2—A3
发布后演示拉取	Windows 仓库 localhost:5000 见第六章；Linux 仓库 localhost:15085 见附录 A5—A8
全过程截图汇总	Windows 图 1—11；Linux 图 A1a、A1b 及 A2—A9，均嵌入本 Word
Docker 环境部署与验证	沿用已装引擎；Windows 启动验证见 2.2，Linux 资源创建及检查见附录 A1

二 实验环境与项目结构

第二至九章记录原 Windows 实验。v3 的 Linux 补充部署使用独立资源，环境配置和实际操作截图见附录 A。

2.1 实验配置与项目目录

项目	本次实际配置
操作系统	Windows 11 家庭版 中文版 64 位，10.0.26300
本地开发工具	Java 21.0.11；Node.js 24.14.1；npm 11.11.0；Maven 3.9.16
应用技术栈	Vue 3.5.44、Vite 7.3.7、Spring Boot 4.1.1、SQLite JDBC 3.53.4.0
Docker 环境	Docker Desktop 4.95.0；Client / Server 29.8.2；Compose 5.6.0
构建与运行镜像	Node 24；Maven 3.9.9 + Java 21；Temurin 21 JRE
源码目录	Code/memory-game6085/
应用端口	宿主机 127.0.0.1:18085 → 容器 8080
镜像仓库	本机 Registry，127.0.0.1:5000
持久化目录	memory-game6085-data → /app/data，数据库 garden.db

frontend 保存 Vue 页面和图案，backend 保存 Spring Boot 接口及数据库逻辑，scripts 保存本地构建与实验辅助脚本。Dockerfile 与 compose.yaml 位于项目根目录。8080 被已有课堂项目占用，因此本实验采用 18085 作为宿主机端口。

2.2 环境启动与验证

本实验沿用 Windows 上已安装的 Docker Desktop，使用 WSL2 的 Linux 容器引擎。原有 WSL 数据目录继续使用。表中的环境版本取自本次实验机器，图 2 展示恢复启动后的实际验证结果。

启动时遇到遗留 socket 故障，Docker Desktop 无法正常提供服务。为恢复启动，在 Docker Desktop 停止时执行下面的辅助脚本，仅为该子进程设置独立的 LOCALAPPDATA 运行时缓存目录，保留系统级环境变量。

```
.\scripts\start-docker-runtime.ps1
```

引擎启动后，在项目根目录执行版本与命名检查。docker version 同时返回 Client 和 Server 29.8.2，Engine 为 linux/amd64；Compose 返回 5.6.0，项目目录为 memory-game6085。随后执行第五章的构建与运行操作。

原 Windows 环境沿用既有安装。本报告附录 A 补充本轮 Linux 环境的网络、数据卷、镜像及容器配置操作截图，两次环境的记录分别保留。

```
docker version

docker compose version

Get-Item . | Select-Object Name, FullName
```

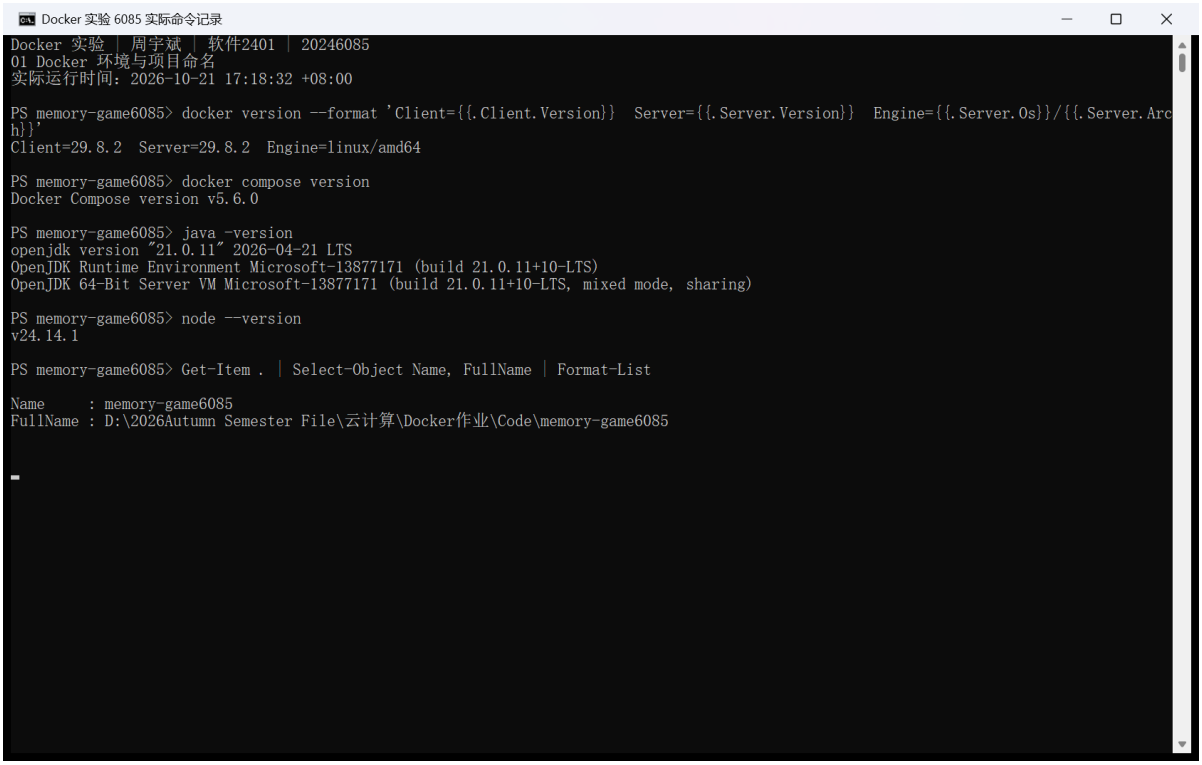


图 2　Docker Client 与 Server 版本及项目名称检查

三 系统设计与持久化

3.1 应用结构

浏览器通过 Vue3 页面操作游戏，Spring Boot 提供牌局、翻牌、暂停、成绩提交和排行榜接口。前端构建结果复制到后端静态资源目录，再打入可执行 JAR，因此一个应用容器即可同时提供页面和接口。

游戏提供轻松 6 对、标准 8 对和挑战 12 对三种难度。后端随机生成卡牌顺序并判断配对，负责步数和计时。未翻开的答案不会直接传给浏览器；暂停时保留本局状态并停止计时，恢复后继续游戏。

3.2 SQLite 数据设计

字段	用途
game_id	牌局唯一标识，防止同一局重复写入
nickname / difficulty	玩家昵称与难度，按难度分别查询排行榜
moves / duration_ms	服务端生成的步数与用时，作为排序依据
created_at	成绩记录创建时间

运行中的牌局保存在服务端内存中；服务重启后需要重新开局。已提交成绩保存在 SQLite 数据库，排行榜按步数升序排列，步数相同再比较用时。持久化对象是已保存成绩，不包括未完成牌局。

3.3 容器数据卷

```
volumes:
  - garden-data:/app/data
# 命名数据卷的实际名称
name: memory-game6085-data
```

镜像保存应用程序，数据库由运行时生成并写入 /app/data。该目录挂载到命名数据卷后，删除应用容器不会自动删除成绩文件。实验重建容器时继续挂载同一数据卷，以验证程序与数据的生命周期可以分离。

四 本地构建与功能验证

4.1 本地构建与自动验证

```
.\scripts\build.ps1
java -jar backend/target/memory-game6085.jar
```

本地构建已生成可执行 JAR。后端测试覆盖隐藏牌面、无效操作、暂停状态、服务端成绩统计和 SQLite 重新打开后的数据保留。浏览器测试覆盖完整配对、昵称校验、成绩提交、榜单刷新、窄屏布局以及接口失败后的重试。

检查项目	结果
npm run build	成功生成前端静态资源
后端测试	3 项通过，0 失败，0 错误
浏览器端到端测试	3 项全部通过
界面检查	1440、768、375、812 横屏及 320 宽度无横向溢出
数据保存	完成牌局后可提交昵称并进入对应难度榜单

4.2 容器中的真实游戏操作

在实际应用容器中通过浏览器完成轻松难度一局，保存昵称“6085 实验”。本次演示记录为 12 步、6.95 秒，记录编号 1。这些数值来自服务端实际返回，仅用于展示成绩保存流程。

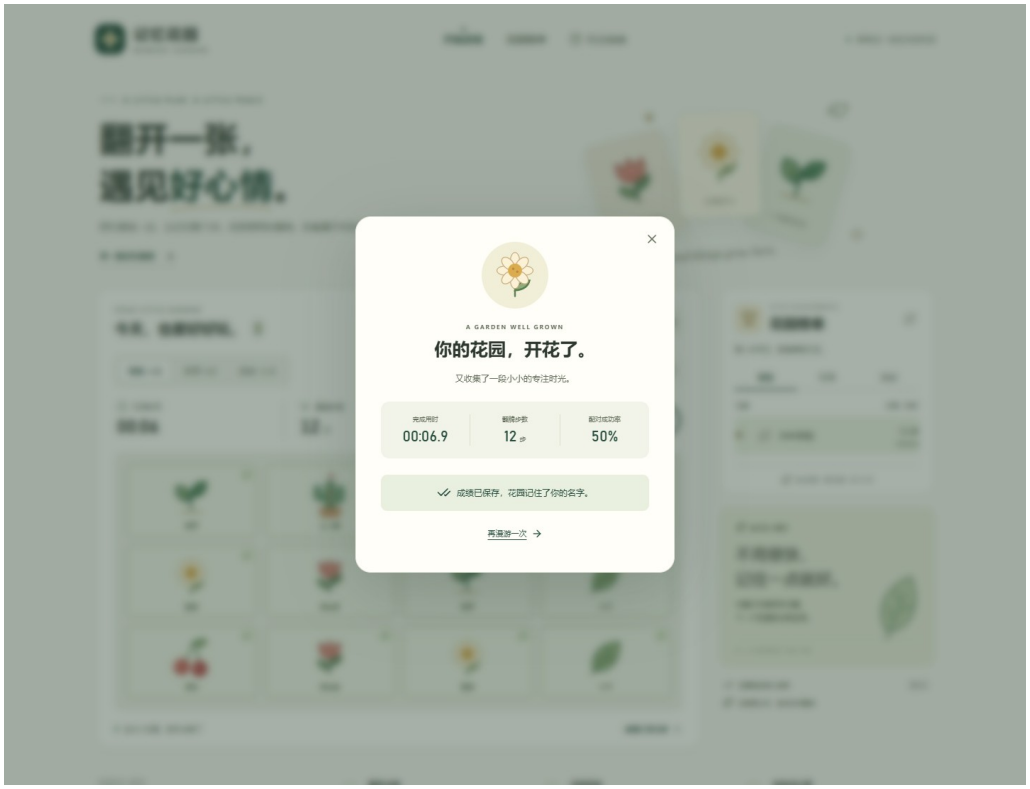


图 3　容器中完成配对并保存演示成绩

五 Docker 镜像构建与运行

5.1 多阶段构建

第一阶段使用 Node 构建 Vue，第二阶段使用 Maven 编译和打包 Spring Boot，第三阶段只复制 JAR 到 Java 运行环境。以下为实际 Dockerfile 的关键流程；完整文件位于项目根目录。

```
FROM node:24-bookworm-slim AS frontend
WORKDIR /build/frontend
COPY frontend/package*.json ./
RUN npm ci --no-audit --no-fund
COPY frontend/ ./
RUN npm run build
FROM maven:3.9.9-eclipse-temurin-21 AS backend
WORKDIR /build/backend
COPY backend/pom.xml ./
RUN mvn -B -ntp dependency:go-offline
COPY backend/src ./src
COPY --from=frontend /build/frontend/dist /build/frontend/dist
RUN mvn -B -ntp package
FROM eclipse-temurin:21-jre-jammy
# 创建普通用户与数据目录后复制 JAR
COPY --from=backend --chown=garden:garden /build/backend/target/memory-game6085.jar
/app/app.jar
USER garden
ENV DATA_DIR=/app/data PORT=8080
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=70.0", "-jar", "/app/app.jar"]
```

5.2 JAR 打包与镜像导出

```
docker compose --progress plain build
```

构建日志显示 Maven BUILD SUCCESS，最终镜像命名为 memory-game6085:1.0.0。Docker CLI 显示磁盘占用约 469 MB。首次基础镜像下载曾发生短读错误，完成基础镜像拉取后重新构建成功。

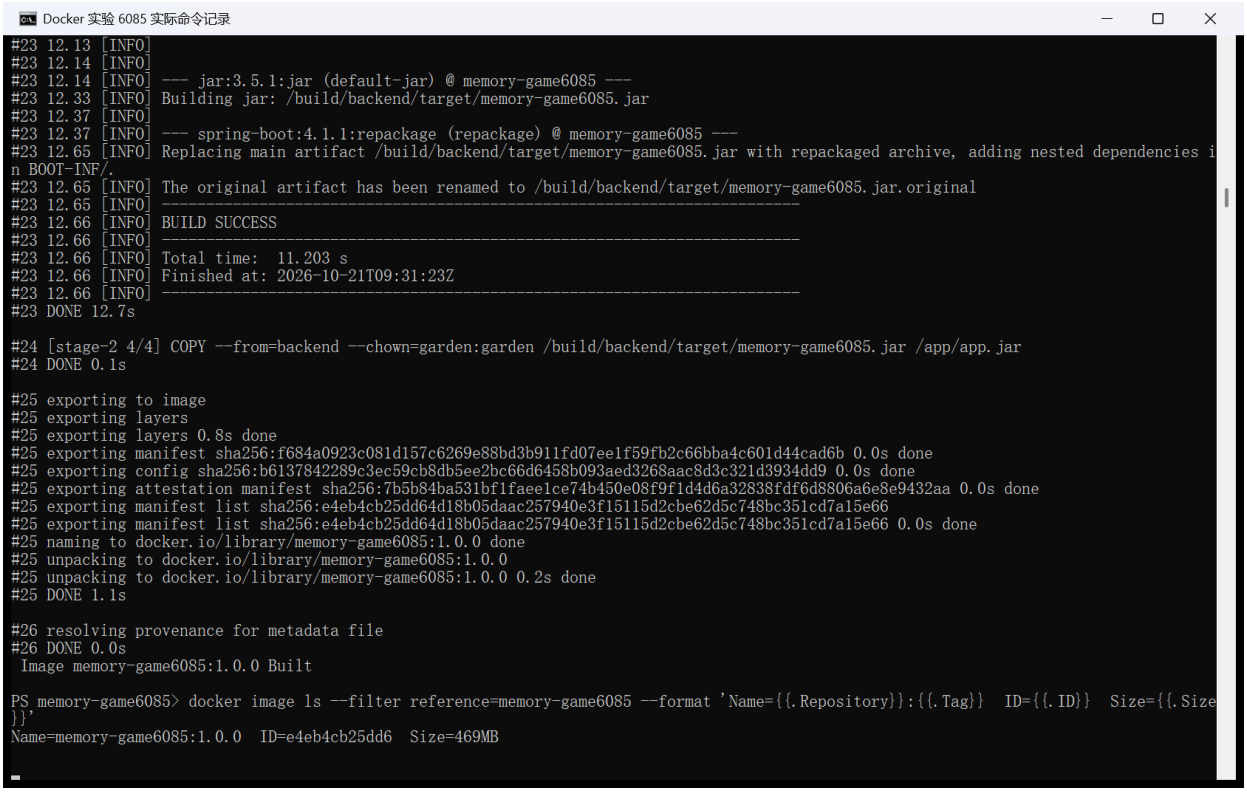


图 4　Spring Boot JAR 打包成功并导出 Docker 镜像

5.3 启动容器与接口检查

```
docker compose up -d --no-build
docker compose ps
curl.exe -s http://localhost:18085/api/health
docker exec memory-game6085-garden-1 id
```

应用容器运行后，18085 端口可访问游戏和 API。健康检查返回 status 为 UP，database 为 SQLite。容器内运行身份为 garden，UID 和 GID 均为 999，应用不使用 root 用户运行。

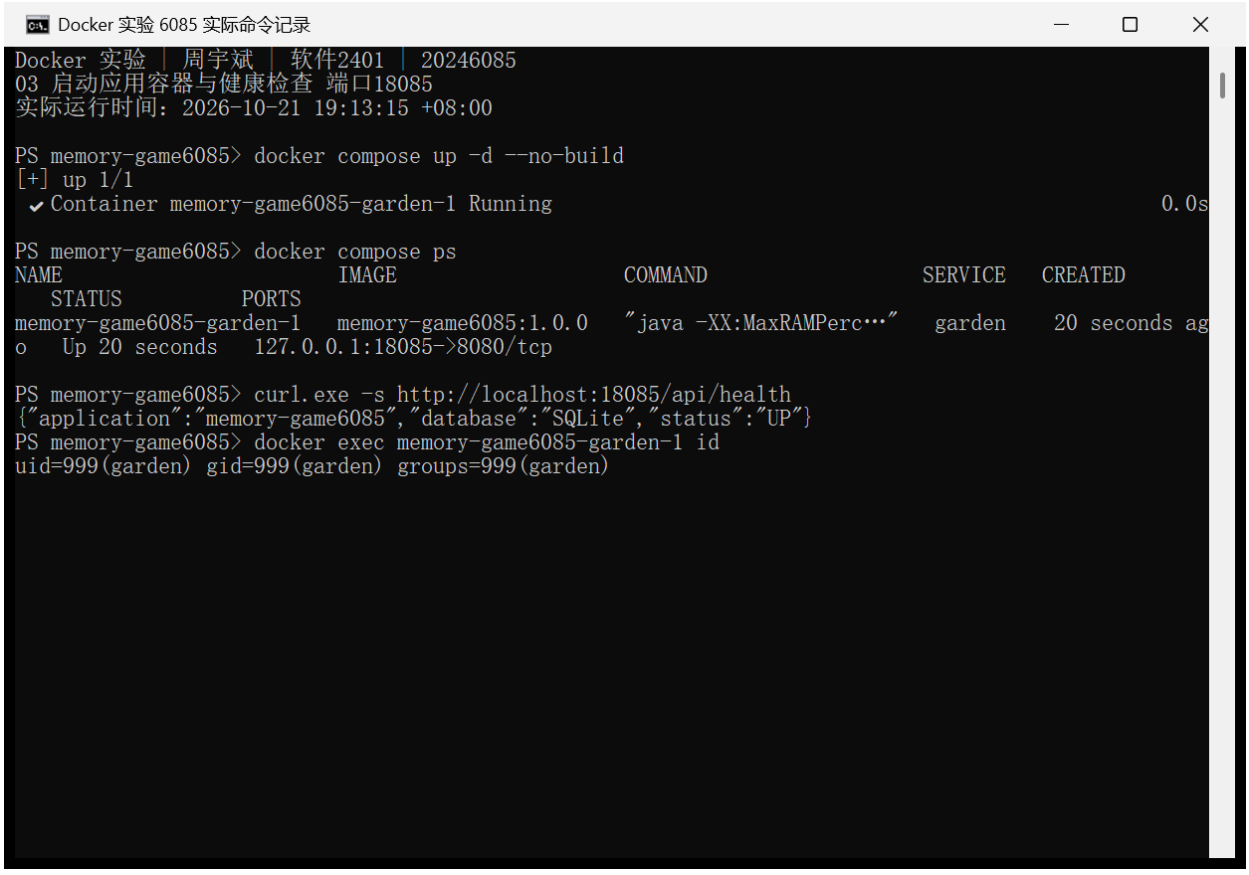


图 5　应用容器运行状态 端口映射 健康检查与普通用户

浏览器访问 http://localhost:18085 可进入游戏。该地址映射至应用容器的 8080，前端和后端同源访问，SQLite 无需额外数据库容器。

六 镜像发布与拉取

6.1 启动本机镜像仓库

本实验使用本机私有 Registry，仓库端口绑定 127.0.0.1:5000，镜像数据保存在 memory-game6085-registry-data 数据卷。仓库用于本机课程实验。

```
docker run -d --name memory-game6085-registry -p 127.0.0.1:5000:5000 -e
OTEL_TRACES_EXPORTER=none --mount type=volume,source=memory-game6085-registry-
data,target=/var/lib/registry registry:3
```

6.2 标记与推送镜像

```
docker tag memory-game6085:1.0.0 localhost:5000/memory-game6085:1.0.0
docker push localhost:5000/memory-game6085:1.0.0
curl.exe -s http://localhost:5000/v2/memory-game6085/tags/list
```

推送日志显示镜像层 Pushed，仓库标签查询返回 1.0.0。发布路径为 localhost:5000/memory-game6085:1.0.0。

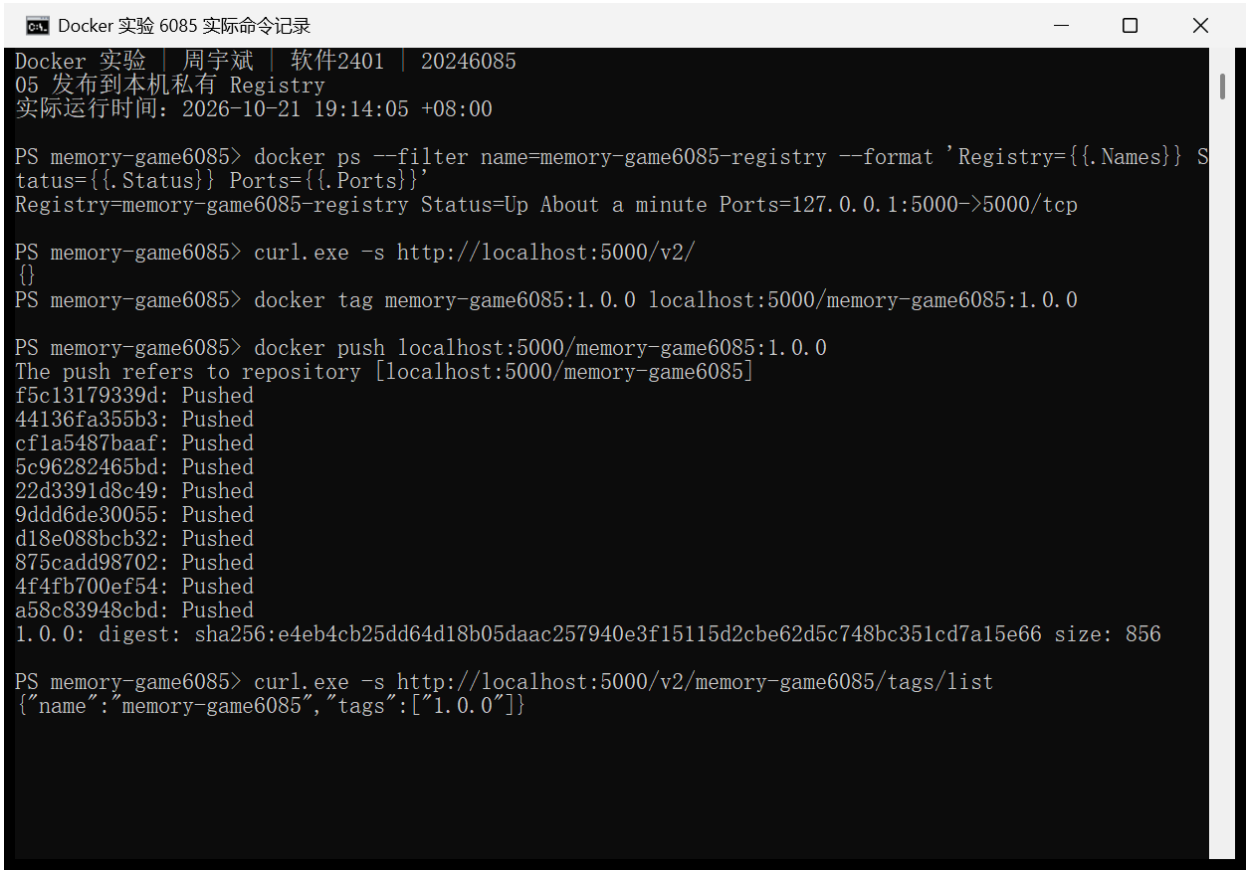


图 6　镜像实际推送成功与仓库标签查询

6.3 删除本地镜像后重新拉取

为验证镜像来自仓库，先移除本实验应用容器，保留数据卷，并删除本地的两个实验镜像标签。镜像列表确认本地不存在 memory-game6085 后，再执行 pull。

```
docker compose down
docker image rm localhost:5000/memory-game6085:1.0.0 memory-game6085:1.0.0
docker image ls
docker pull localhost:5000/memory-game6085:1.0.0
```

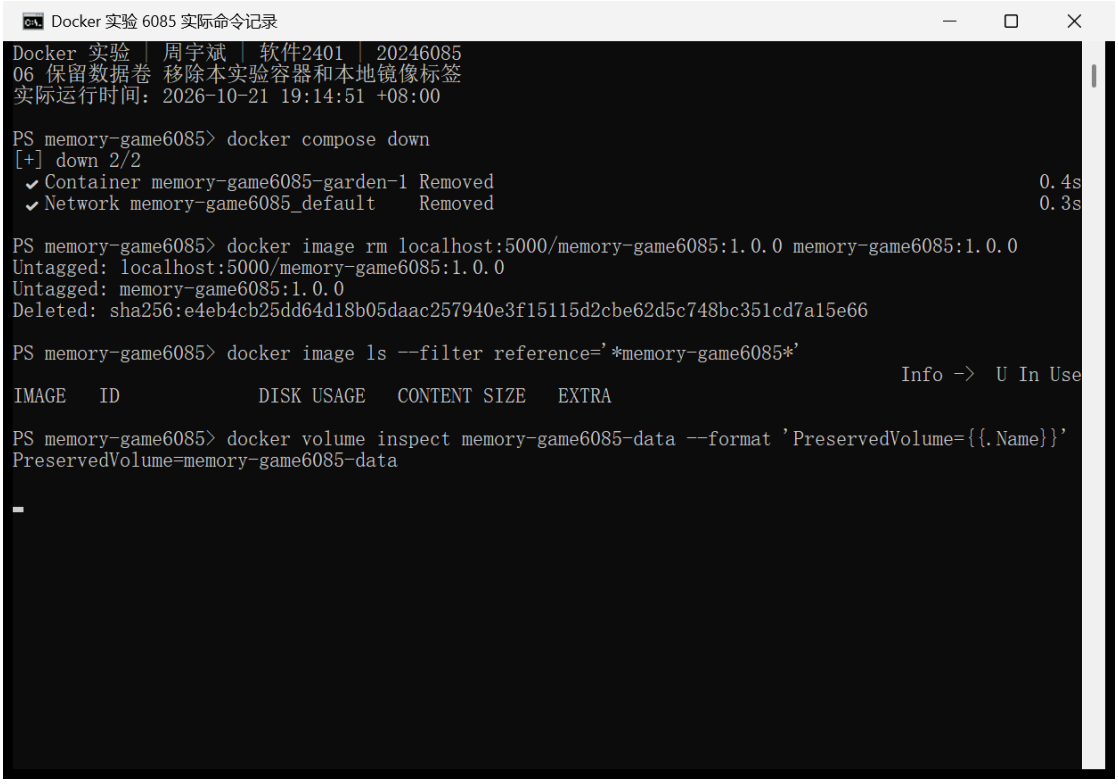


图 7　移除实验容器和本地镜像 数据卷仍然保留

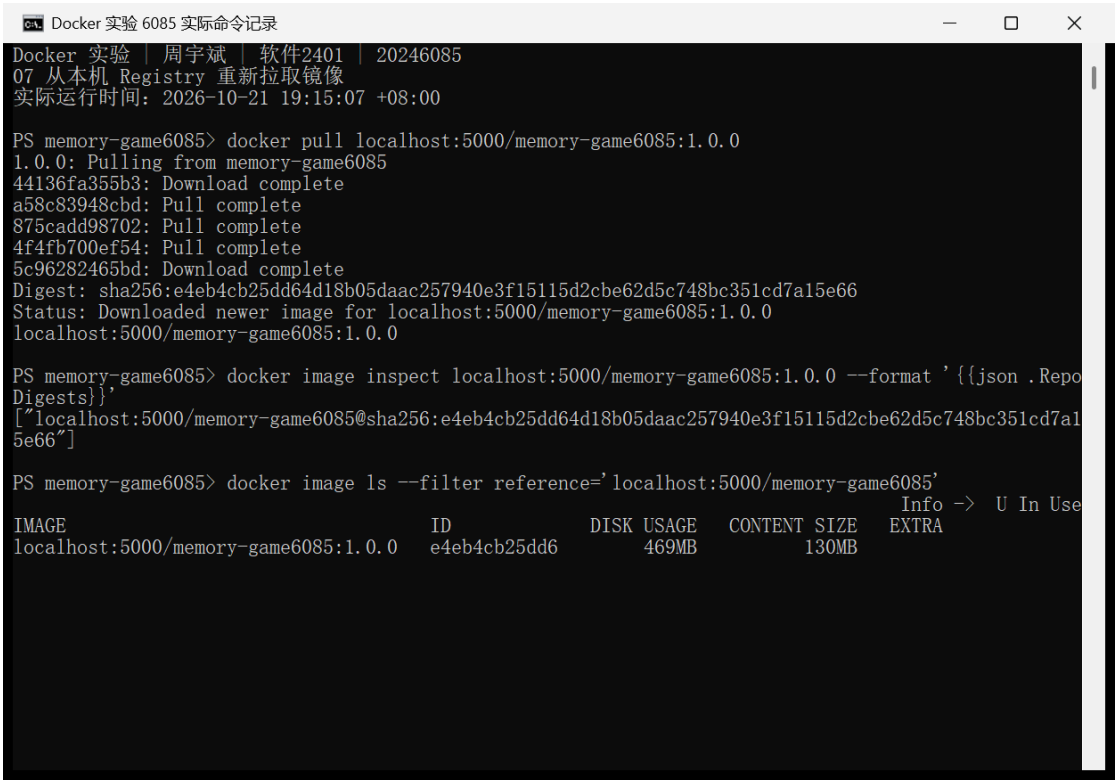


图 8　从 Registry 重新拉取成功

拉取结果为 Downloaded newer image。推送与拉取的摘要一致，说明使用的是已发布的同一版本镜像。摘要记录如下：

```
sha256:e4eb4cb25dd64d18b05daac257940e3f15115d2cbe62d5c748bc351cd7a15e66
```

七 容器重建与持久化验证

7.1 记录重建前的数据

原容器中已保存轻松难度演示成绩。实验记录了原容器标识和完整排行榜响应；移除容器时未使用删除数据卷的参数。

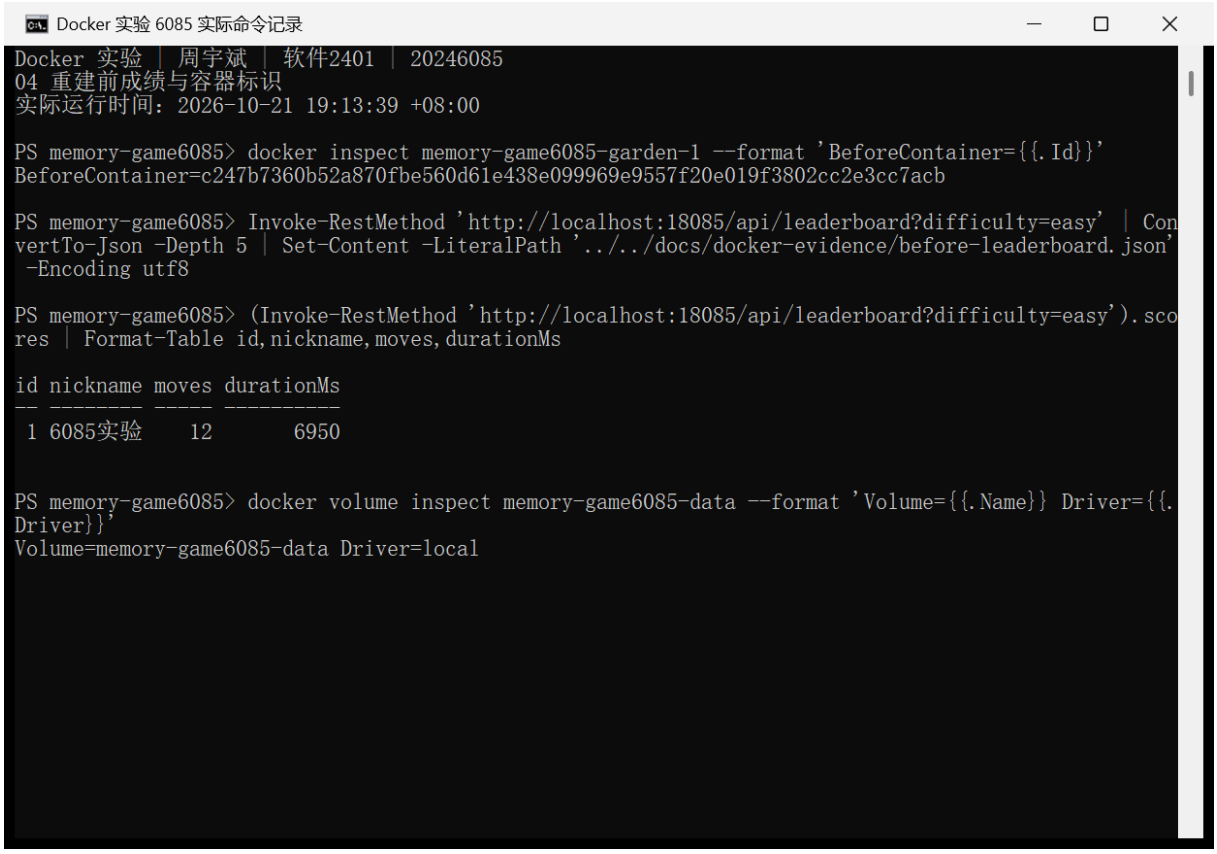


图 9　重建前的容器标识 成绩与数据卷

对比项	重建前的实际值
容器标识前 12 位	c247b7360b52
成绩编号 / 昵称	1 / 6085 实验
步数 / 用时	12 / 6950 ms
数据卷	memory-game6085-data

7.2 使用拉取镜像创建新容器

```
docker run -d --name memory-game6085-pulled -p 127.0.0.1:18085:8080 -e DATA_DIR=/app/data
--mount type=volume,source=memory-game6085-data,target=/app/data localhost:5000/memory-
game6085:1.0.0
```

新容器标识前 12 位为 dd8b4bb17b50，与原容器不同。新容器直接使用拉取的仓库镜像，并继续挂载 memory-game6085-data 到 /app/data。

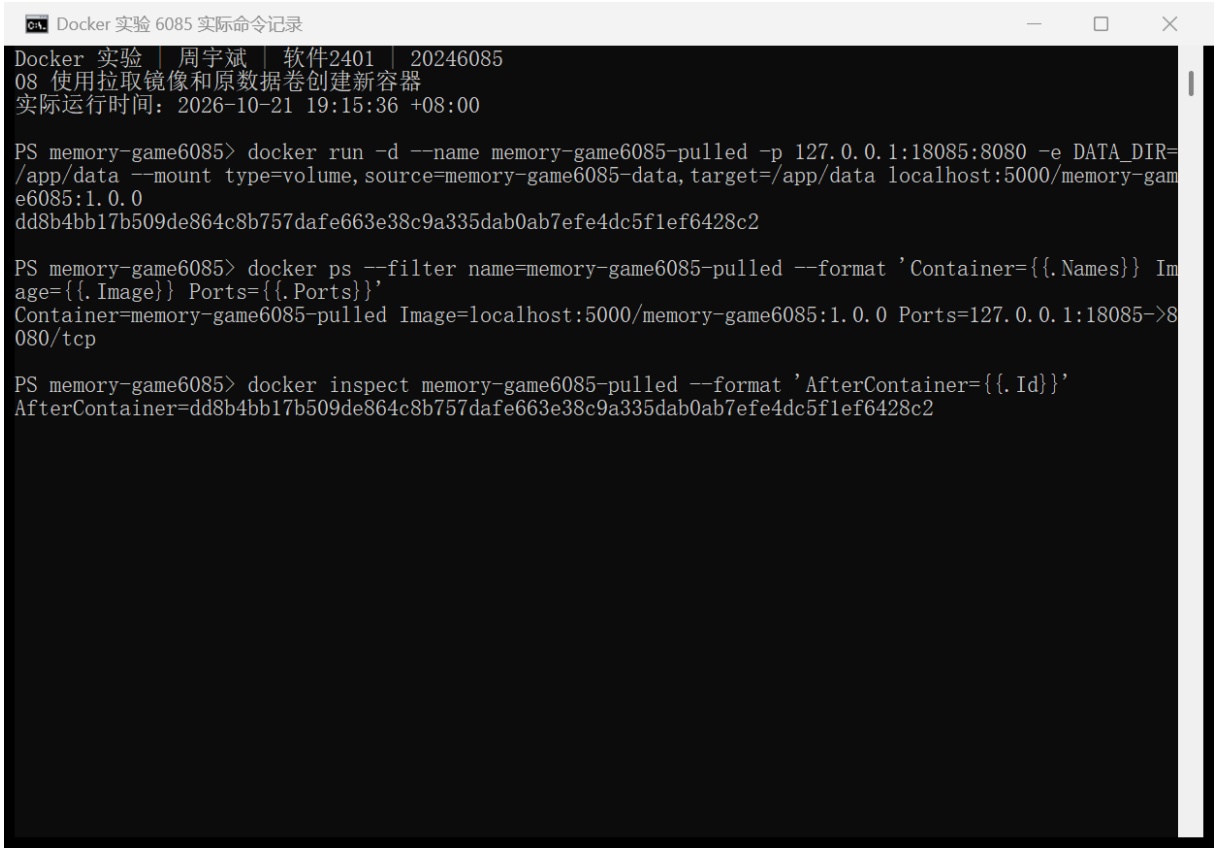


图 10　使用拉取镜像创建新容器并确认容器标识

7.3 对比重建后的成绩

新容器启动后，成绩编号、昵称、难度、步数、用时和创建时间均与重建前一致。程序对重建前后的完整排行榜 JSON 比较，结果为 LeaderboardPreserved=True。记录仍为 6085 实验、12 步、6950 ms。

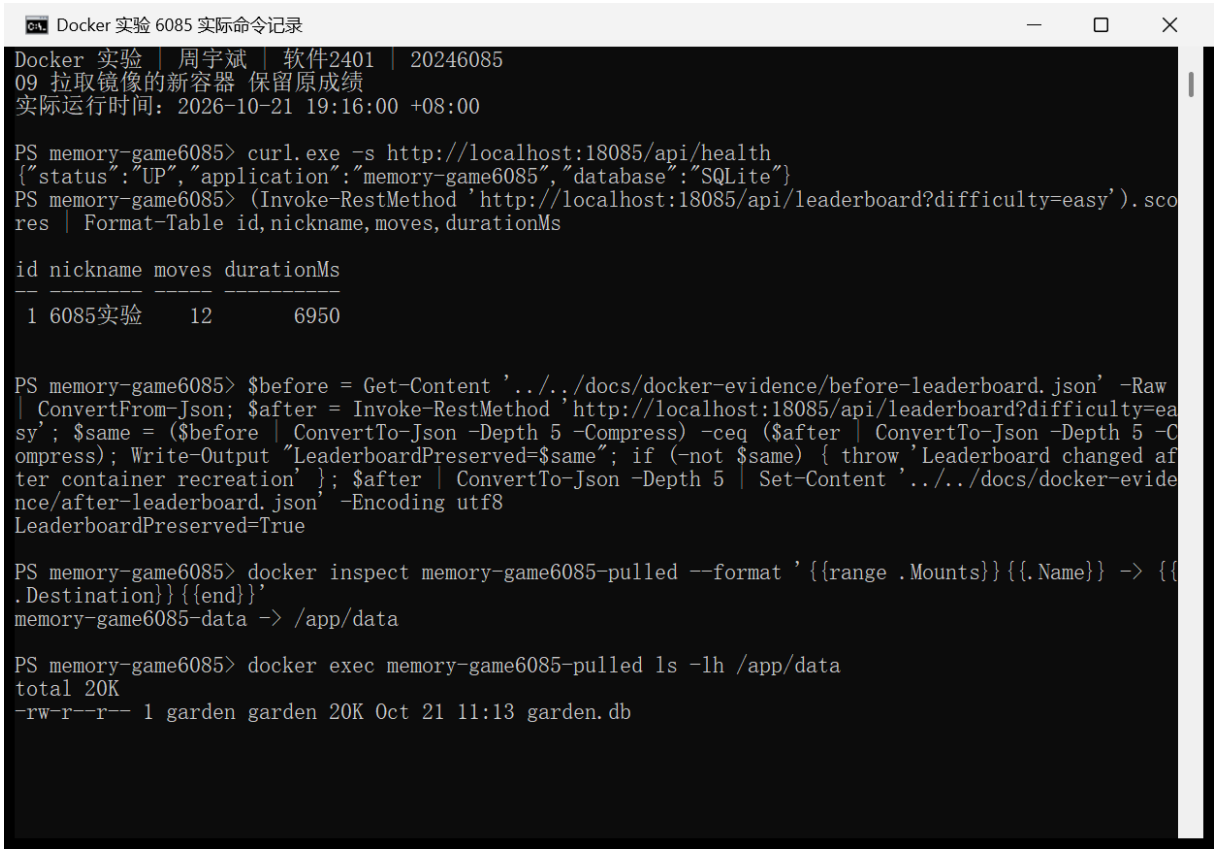


图 11　重建后榜单一致 数据卷仍挂载到同一目录

八 实验结果与问题分析

实验环节	实际结果
Docker 环境	Client / Server 正常，Linux 引擎可用
应用与镜像构建	前端构建、后端测试和 Spring Boot JAR 打包成功
容器运行	页面和 API 可访问，健康检查 UP
镜像发布	本机 Registry push 成功，可查询标签 1.0.0
镜像拉取	删除本地镜像后 pull 成功，摘要与推送一致
容器重建	新容器使用拉取镜像，容器标识与原容器不同
SQLite 数据卷	完整榜单对比一致，保存成绩保留
实验截图	真实终端窗口与容器浏览器页面均已采集

8.1 启动与端口问题

Docker Desktop 最初因遗留 socket 无法访问而退出。原运行时目录改名被 Windows 拒绝，最终通过仅为 Docker Desktop 子进程设置独立运行时缓存目录恢复启动，系统级环境变量未修改。恢复后以 Client 与 Server 响应以及实际容器结果验证环境可用。

首次构建遇到基础镜像下载中断；重新拉取所需基础镜像后完成构建。现有课堂项目占用 8080，因此将本实验宿主机端口调整为 18085，同时保留容器内部端口 8080。

8.2 实验总结

本实验完成了应用打包、镜像构建、容器访问、镜像发布与拉取，以及数据卷跨容器保留的完整验证。镜像用于分发应用程序，容器是运行实例，Registry 用于保存和分发镜像，数据卷用于保存独立于容器生命周期的数据库文件。

重建前后容器标识发生变化，但保存的成绩没有变化，说明 SQLite 文件与应用容器已经通过命名数据卷实现生命周期分离。后续复现实验时，应保持仓库地址、镜像版本、应用端口和数据卷名称一致。

九 参考资料与证据来源

原始作业要求：raw/要求.png。实验项目为记忆花园 memory-game6085，采用 Vue3、Spring Boot 和 SQLite，镜像仓库为本机 Registry。

项目源代码：Code/memory-game6085/；实验结果：docs/docker-evidence/results.json；各图来自同目录的真实终端和浏览器截图。

技术资料查阅日期：2026 年 10 月 21 日。

Vue 官方文档：https://vuejs.org/guide/essentials/reactivity-fundamentals.html

Vite 官方文档：https://vite.dev/guide/

Spring Boot 官方文档：https://docs.spring.io/spring-boot/maven-plugin/packaging.html

SQLite JDBC 官方仓库：https://github.com/xerial/sqlite-jdbc

Docker 数据卷官方文档：https://docs.docker.com/engine/storage/volumes/

CNCF Distribution 官方部署文档：https://distribution.github.io/distribution/about/deploying/

9.1 原 Windows 实验截图来源索引

图号	内容	来源文件
1	容器首页	04-container-home.png
2	环境与项目命名	01-environment.png
3	容器中保存成绩	05-container-completed.png
4	Spring Boot JAR 打包与镜像导出	02-build-progress.png
5	容器运行与健康检查	03b-container-run.png
6	本机仓库推送	05-registry-push.png
7	本地镜像移除	06-remove-local.png
8	从仓库重新拉取	07-registry-pull.png
9	重建前成绩	04-data-before.png
10	拉取镜像创建新容器	08-run-pulled-image.png
11	重建后榜单与数据卷	09-data-after.png

截图源文件统一位于 docs/docker-evidence/。终端图片采集自专用 Windows 控制台窗口，浏览器图片采集自实际运行的应用容器。完整实验标识、镜像摘要和成绩对比保存在同目录 results.json。

附录 A Linux 补充部署与复现

A1 环境来源与独立资源配置

2026 年 10 月 21 日在 Debian GNU/Linux 13 云工作区补做部署。平台已提供 Docker Engine，本轮沿用其本地守护进程；Client 与 Server 均为 28.4.0，Compose 为 2.40.3，存储驱动为 vfs。

实际创建独立网络 memory-game6085-v3-net，以及应用和仓库数据卷。图 A1a 记录首次创建命令及返回结果，图 A1b 记录随后配置检查；完整日志保存在证据包。两次实验分别使用独立资源。

```
docker network create --label course=memory-game6085-v3 \
  memory-game6085-v3-net
docker volume create --label course=memory-game6085-v3 \
  memory-game6085-v3-data
docker volume create --label course=memory-game6085-v3 \
  memory-game6085-v3-registry-data
```



图 A1a 首次创建独立网络及数据卷

资源创建后的配置检查

创建资源后，分别检查引擎与 Compose 版本、网络驱动和应用及仓库数据卷。检查返回 network 为 bridge、volume 为 local，与首次创建结果对应。

```
Linux v3 supplementary experiment - platform Docker reused
+ date -u +%Y-%m-%dT%H:%M:%SZ
2026-10-21T13:28:36Z
+ docker version --format 'Client={{.Client.Version}} Server={{.Server.Version}}'
Client=28.4.0 Server=28.4.0
+ docker info --format 'OS={{.OperatingSystem}}; Driver={{.Driver}}'
OS=Debian GNU/Linux 13 (trixie) (containerized); Driver=vfs
+ docker compose version
Docker Compose version v2.40.3

Isolated resources created in 01-linux-config.txt; configuration verification:
+ docker network inspect memory-game6085-v3-net --format 'Network={{.Name}} Driver={{.Driver}}'
Network=memory-game6085-v3-net Driver=bridge
+ docker volume inspect memory-game6085-v3-data --format 'Volume={{.Name}} Driver={{.Driver}}'
Volume=memory-game6085-v3-data Driver=local
+ docker volume inspect memory-game6085-v3-registry-data --format 'Volume={{.Name}} Driver={{.Driver}}'
Volume=memory-game6085-v3-registry-data Driver=local

Original Windows installation history is separate from this Linux experiment.

[Live stage finished; press Enter to close]
```

图 A1b Linux 引擎及网络和数据卷配置验证

两轮实验均沿用已安装的 Docker 引擎。本附录记录当前环境的资源配置与应用部署；本报告未附原 Windows 安装、首次配置和启动恢复的历史操作截图，现有记录不包含引擎安装全过程。

A2 配套 JAR 打包为新镜像

使用 Dockerfile.prebuilt 将配套可执行 JAR 复制到 Temurin 21 JRE 镜像，配置普通用户 garden 和数据目录/app/data。此步骤使用已完成 Maven 打包的交付 JAR；源码多阶段构建流程仍见第五章。

交付 JAR 的 SHA256

```
2e97ff46f2acfc1bfe0411a22f3f61a3ef4b8958e4ba5fff609d0350853852a0
```

```
docker build --progress=plain --pull=false -f Dockerfile.prebuilt -t memory-game6085:1.0.1-v3 .
```

```
Linux v3 - repeatable image build from the exact deliverable JAR
+ date -u +%Y-%m-%dT%H:%M:%SZ
2026-10-21T13:28:47Z
+ sha256sum -c dist-artifacts/SHA256SUMS
dist-artifacts/memory-game6085.jar: OK
+ docker build --progress=plain --pull=false -f Dockerfile.prebuilt -t memory-game6085:1.0.1-v3 .
#0 building with "default" instance using docker driver

#1 [internal] load build definition from Dockerfile.prebuilt
#1 transferring dockerfile: 528B done
#1 DONE 0.0s

#2 [internal] load metadata for docker.io/library/eclipse-temurin:21-jre-jammy
#2 DONE 0.0s

#3 [internal] load .dockerignore
#3 transferring context: 145B done
#3 DONE 0.0s

#4 [1/4] FROM docker.io/library/eclipse-temurin:21-jre-jammy
#4 DONE 0.0s

#5 [internal] load build context
#5 transferring context: 92B done
#5 DONE 0.0s

#6 [3/4] WORKDIR /app
#6 CACHED

#7 [2/4] RUN groupadd --system garden && useradd --system --gid garden --home-dir /app garden     && mkdir -p /app/data && chown -R garden:g
arden /app
#7 CACHED

#8 [4/4] COPY --chown=garden:garden dist-artifacts/memory-game6085.jar /app/app.jar
#8 CACHED

#9 exporting to image
#9 exporting layers done
#9 writing image sha256:cd0d87b180cc1916dea642a7d1d5b62988c943de635577dd44f1d0229aae43d9 done
#9 naming to docker.io/library/memory-game6085:1.0.1-v3 done
#9 DONE 0.0s
+ docker image inspect memory-game6085:1.0.1-v3 --format 'ImageID={{.Id}} User={{.Config.User}}'
ImageID=sha256:cd0d87b180cc1916dea642a7d1d5b62988c943de635577dd44f1d0229aae43d9 User=garden

[Live stage finished; press Enter to close]
```

图 A2 校验交付 JAR 并成功构建 1.0.1-v3 镜像

A3 容器运行与交付文件校验

将镜像运行在独立网络中，宿主机 127.0.0.1:18086 映射容器 8080，并挂载 memory-game6085-v3-data。健康接口返回 UP，进程身份为 garden。

分别计算交付 JAR 与容器/app/app.jar 的 SHA256，两个结果完全一致。这将本轮交付文件与实际部署镜像直接对应。

关键命令摘自 03-container-run.txt。以下文本便于复制，实际返回结果见原图。

```
docker run -d --name memory-game6085-v3-app \
  --network memory-game6085-v3-net \
  -p 127.0.0.1:18086:8080 -e DATA_DIR=/app/data \
  --mount type=volume,source=memory-game6085-v3-data,target=/app/data \
  memory-game6085:1.0.1-v3
curl --fail --silent http://127.0.0.1:18086/api/health
docker exec memory-game6085-v3-app sha256sum /app/app.jar
```

交付 JAR 与容器内文件 SHA256 相同

```
2e97ff46f2acfc1bfe0411a22f3f61a3ef4b8958e4ba5fff609d0350853852a0
```

```
Linux v3 - run the new image with isolated network and data volume
+ date -u +%Y-%m-%dT%H:%M:%SZ
2026-10-21T13:28:55Z
+ docker run -d --name memory-game6085-v3-app --network memory-game6085-v3-net -p 127.0.0.1:18086:8080 -e DATA_DIR=/app/data --mount type=vo
lume,source=memory-game6085-v3-data,target=/app/data memory-game6085:1.0.1-v3
263d322a4f4c7b1a2a211d61429d0e6ac18fb5e5fa77c5fb4f783871004577d1
+ docker ps --filter name=memory-game6085-v3-app --format 'Name={{.Names}} Ports={{.Ports}}'
Name=memory-game6085-v3-app Ports=127.0.0.1:18086->8080/tcp
+ curl --fail --silent http://127.0.0.1:18086/api/health
{"application":"memory-game6085","database":"SQLite","status":"UP"}
+ docker exec memory-game6085-v3-app id
uid=999(garden) gid=999(garden) groups=999(garden)
+ sha256sum /workspace/DockerHomework/Code/memory-game6085/dist-artifacts/memory-game6085.jar
2e97ff46f2acfc1bfe0411a22f3f61a3ef4b8958e4ba5fff609d0350853852a0  /workspace/DockerHomework/Code/memory-game6085/dist-artifacts/memory-game6
085.jar
+ docker exec memory-game6085-v3-app sha256sum /app/app.jar
2e97ff46f2acfc1bfe0411a22f3f61a3ef4b8958e4ba5fff609d0350853852a0  /app/app.jar
+ docker inspect memory-game6085-v3-app --format 'Container={{.Id}} Volume={{(index .Mounts 0).Name}}'
Container=263d322a4f4c7b1a2a211d61429d0e6ac18fb5e5fa77c5fb4f783871004577d1 Volume=memory-game6085-v3-data

[Live stage finished; press Enter to close]
```

图 A3 新容器运行成功且容器内外 JAR 摘要相同

A4 实际游戏与成绩保存

通过浏览器在本轮 Linux 容器完成轻松难度牌局，并提交昵称“6085v3 实验”。实际记录编号为 1，11 步，用时 7268 ms；数据由服务端生成并保存到 SQLite。

本轮同时采集 1440 宽桌面和 375 宽手机页面，手机页面未出现横向溢出。配套预览图片来自本次容器访问。

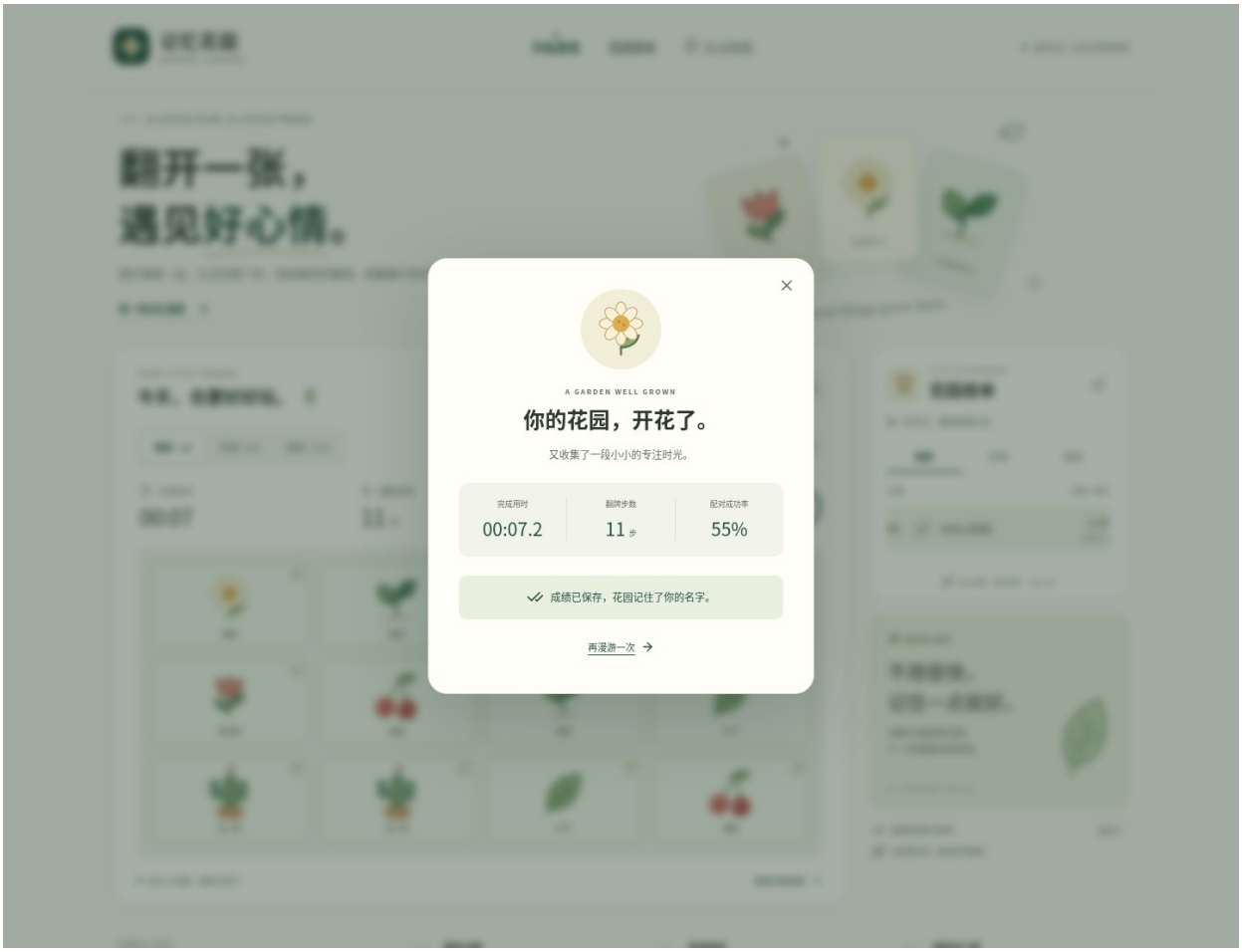


图 A4 Linux 容器内完成牌局并实际保存成绩

A5 本机仓库发布镜像

启动独立 Registry 容器，绑定 127.0.0.1:15085，并使用专用仓库数据卷。将新镜像标记到 localhost:15085 后执行 push，仓库标签查询返回 1.0.1-v3。此地址仅用于本机实验。

关键命令及结果摘自 04-registry-push.txt。

```
docker tag memory-game6085:1.0.1-v3 \
  localhost:15085/memory-game6085:1.0.1-v3
docker push localhost:15085/memory-game6085:1.0.1-v3
curl --fail --silent \
  http://127.0.0.1:15085/v2/memory-game6085/tags/list
```

推送与后续拉取的仓库摘要一致

```
sha256:fe814da12f1ebd9dbbfc85dcab5af55d9ef098b9261bc706f0df362746f78c85
```

```
Linux v3 - isolated local Registry and actual image publication
+ date -u +%Y-%m-%dT%H:%M:%SZ
2026-10-21T13:30:35Z
+ docker run -d --name memory-game6085-v3-registry --network memory-game6085-v3-net -p 127.0.0.1:15085:5000 -e OTEL_TRACES_EXPORTER=none --m
ount type=volume,source=memory-game6085-v3-registry-data,target=/var/lib/registry registry:3
Unable to find image 'registry:3' locally
3: Pulling from library/registry
e2de96513ba9: Pull complete
90764beeca7f: Pull complete
2aef3a8beb68: Pull complete
783ed5b12fca: Pull complete
8dc2188d2a74: Pull complete
Digest: sha256:ddf754342cfc8acc51a56d5d0ab6af068264618644606360d8bd5c546dab2a7b8
Status: Downloaded newer image for registry:3
af80d5f94b1f997bae5288ee585791314836350ba7e2a6094e0a41d6941527fe
+ docker tag memory-game6085:1.0.1-v3 localhost:15085/memory-game6085:1.0.1-v3
+ docker push localhost:15085/memory-game6085:1.0.1-v3
The push refers to repository [localhost:15085/memory-game6085]
9fe6d5aebac8: Pushed
5f70bf18a086: Pushed
b368e8d29060: Pushed
2302df99c4d0: Pushed
82b978d0aab5: Pushed
98bd9880f810: Pushed
34af552be181: Pushed
129f77d88403: Pushed
1.0.1-v3: digest: sha256:fe814da12f1ebd9dbbfc85dcab5af55d9ef098b9261bc706f0df362746f78c85 size: 1994
+ curl --fail --silent http://127.0.0.1:15085/v2/memory-game6085/tags/list
{"name":"memory-game6085","tags":["1.0.1-v3"]}

+ docker image inspect localhost:15085/memory-game6085:1.0.1-v3 --format 'RepoDigests={{json .RepoDigests}}'
RepoDigests=["localhost:15085/memory-game6085@sha256:fe814da12f1ebd9dbbfc85dcab5af55d9ef098b9261bc706f0df362746f78c85"]

[Live stage finished; press Enter to close]
```

图 A5 启动本机 Registry 并实际推送新镜像

A6 移除本地镜像并保留数据卷

在发布成功并记录完整排行榜后，移除本轮应用容器和两个 1.0.1-v3 本地标签。按版本过滤的镜像列表为空；命名数据卷仍存在。移除操作仅针对本轮独立资源。

```
Linux v3 - remove only v3 application container and both local image tags
+ date -u +%Y-%m-%dT%H:%M:%SZ
2026-10-21T13:32:36Z
+ docker rm -f memory-game6085-v3-app
memory-game6085-v3-app
+ docker image rm localhost:15085/memory-game6085:1.0.1-v3 memory-game6085:1.0.1-v3
Untagged: localhost:15085/memory-game6085:1.0.1-v3
Untagged: localhost:15085/memory-game6085@sha256:fe814da12f1ebd9dbbfc85dcab5af55d9ef098b9261bc706f0df362746f78c85
Untagged: memory-game6085:1.0.1-v3
Deleted: sha256:cd0d87b180cc1916dea642a7d1d5b62988c943de635577dd44f1d0229aae43d9
+ docker image ls --filter 'reference=*memory-game6085*1.0.1-v3' --format '{{.Repository}}:{{.Tag}} {{.ID}}'
No v3 image rows above. Named data volume is retained:
+ docker volume inspect memory-game6085-v3-data --format '{{.Name}} {{.Driver}}'
memory-game6085-v3-data local

[Live stage finished; press Enter to close]
```

图 A6 应用容器及本地镜像已移除而数据卷保留

A7 从仓库重新拉取

本地版本移除后执行 docker pull，结果为 Downloaded newer image。推送与拉取的仓库摘要一致，与原 Windows 实验的摘要分别记录。

Linux 补充实验仓库摘要

sha256:fe814da12f1ebd9dbbfc85dcab5af55d9ef098b9261bc706f0df362746f78c85

```
Linux v3 - pull the published version after local image removal
+ date -u +%Y-%m-%dT%H:%M:%SZ
2026-10-21T13:32:49Z
+ docker pull localhost:15085/memory-game6085:1.0.1-v3
1.0.1-v3: Pulling from memory-game6085
dc541b8d0272: Already exists
f5c13179339d: Already exists
22d3391d8c49: Already exists
9ddd6de30055: Already exists
d18e088bcb32: Already exists
a7cc26c8ce18: Already exists
4f4fb700ef54: Already exists
38bcc9d2dc56: Already exists
Digest: sha256:fe814da12f1ebd9dbbfc85dcab5af55d9ef098b9261bc706f0df362746f78c85
Status: Downloaded newer image for localhost:15085/memory-game6085:1.0.1-v3
localhost:15085/memory-game6085:1.0.1-v3
+ docker image inspect localhost:15085/memory-game6085:1.0.1-v3 --format 'ImageID={{.Id}} RepoDigests={{json .RepoDigests}}'
ImageID=sha256:cd0d87b180cc1916dea642a7d1d5b62988c943de635577dd44f1d0229aae43d9 RepoDigests=["localhost:15085/memory-game6085@sha256:fe814da
12f1ebd9dbbfc85dcab5af55d9ef098b9261bc706f0df362746f78c85"]

[Live stage finished; press Enter to close]
```

图 A7　本地镜像移除后从 Registry 实际拉取成功

A8 重建容器并核对成绩保留

使用刚拉取的仓库镜像创建新容器，并挂载相同命名数据卷。原容器前 12 位为 263d322a4f4c，新容器前 12 位为 50fb7d90259a；新旧标识不同。

健康检查再次通过，容器内 JAR 仍与交付文件一致，重建前后的完整排行榜 JSON 相同。比较结果为 LeaderboardPreserved=True、ContainerChanged=True。

关键命令及结果摘自 09-recreate-verify.txt；重建继续使用相同数据卷。

```
docker run -d --name memory-game6085-v3-pulled \
  --network memory-game6085-v3-net \
  -p 127.0.0.1:18086:8080 -e DATA_DIR=/app/data \
  --mount type=volume,source=memory-game6085-v3-data,target=/app/data \
  localhost:15085/memory-game6085:1.0.1-v3
docker exec memory-game6085-v3-pulled sha256sum /app/app.jar
```

```
容器内 JAR SHA256
2e97ff46f2acfc1bfe0411a22f3f61a3ef4b8958e4ba5fff609d0350853852a0
LeaderboardPreserved=True ContainerChanged=True
```

```
  Linux v3 - new container from pulled image, same named volume
  + date -u +%Y-%m-%dT%H:%M:%SZ
  2026-10-21T13:33:31Z
  + docker run -d --name memory-game6085-v3-pulled --network memory-game6085-v3-net -p 127.0.0.1:18086:8080 -e DATA_DIR=/app/data --mount type
  =volume,source=memory-game6085-v3-data,target=/app/data localhost:15085/memory-game6085:1.0.1-v3
  50fb7d90259a8f7b690edbd591711676d26021e46deacd99c72682293b5b2c91
  + curl --fail --silent http://127.0.0.1:18086/api/health
  {"database":"SQLite","status":"UP","application":"memory-game6085"}
  + docker inspect memory-game6085-v3-pulled --format 'Container={{.Id}} Image={{.Image}} Volume={{(index .Mounts 0).Name}}'
  Container=50fb7d90259a8f7b690edbd591711676d26021e46deacd99c72682293b5b2c91 Image=sha256:cd0d87b180cc1916dea642a7d1d5b62988c943de635577dd44f1
  d0229aae43d9 Volume=memory-game6085-v3-data
  + docker exec memory-game6085-v3-pulled sha256sum /app/app.jar
  2e97ff46f2acfc1bfe0411a22f3f61a3ef4b8958e4ba5fff609d0350853852a0  /app/app.jar
  + curl --fail --silent 'http://127.0.0.1:18086/api/leaderboard?difficulty=easy'
  {"scores":[{"id":1,"nickname":"6085v3实验","difficulty":"easy","moves":11,"durationMs":7268,"createdAt":"2026-10-21T13:30:47.356448060Z"}],"
  total":1,"bestDurationMs":7268}
  + python /workspace/DockerHomework/docs/v3-evidence/compare-results.py
  LeaderboardPreserved=True ContainerChanged=True
  OldContainer=263d322a4f4c NewContainer=50fb7d90259a

  [Live stage finished; press Enter to close]
```

图 A8 拉取镜像重建后 JAR 及完整排行榜校验通过

A9 浏览器复核重建后的榜单

重新访问本轮应用端口并切换到轻松难度，页面仍显示先前保存的真实成绩。该页面由新容器提供，与终端 JSON 比较结果一致。



图 A9 新容器页面仍显示原已保存成绩

A10 补充证据来源与版本对应

正文图 1 至图 11 沿用原 Windows 实验的真实截图；本附录图 A1a、图 A1b 及图 A2 至图 A9 来自 2026 年 10 月 21 日 Linux 补充实验。终端图是实际终端窗口的像素采集，浏览器图来自当次应用容器。v4 新增图 A1a 并将关键命令整理为可复制文字，原始截图与日志保持原字节。

图号	内容	来源文件
A1a	首次资源创建	01-resource-creation.png
A1b	配置检查	01-config-review.png
A2	JAR 与镜像构建	02-build-review.png
A3	运行与 JAR 一致	03-container-run.png
A4	游戏成绩保存	05-game-saved.png
A5	本机仓库发布	04-registry-push.png
A6	本地镜像移除	07-remove-local.png
A7	仓库拉取	08-registry-pull.png
A8	重建与数据比较	09-recreate-verify.png
A9	重建后页面	10-after-recreate-ui.png

补充证据目录为 docs/v3-evidence，结果文件为 results.json。Windows 原实验镜像为 1.0.0，Linux 补充镜像为 1.0.1-v3；v4 为报告与交付资料修订，未更改应用 JAR、镜像标签或实验结果。两套仓库均仅绑定本机，复现实验时应在实际执行机器上使用对应地址和数据卷。